

TREES, MUTABILITY AND ITERATORS Meta

COMPUTER SCIENCE MENTORS 61A

September 29 – October 6, 2025

Note: Feel free to ask students about what topics they'd like to go through, since we have more questions in the worksheet than usual because many concepts were covered in the last week. **Example Timeline**

- Ice Breaker [5 min]

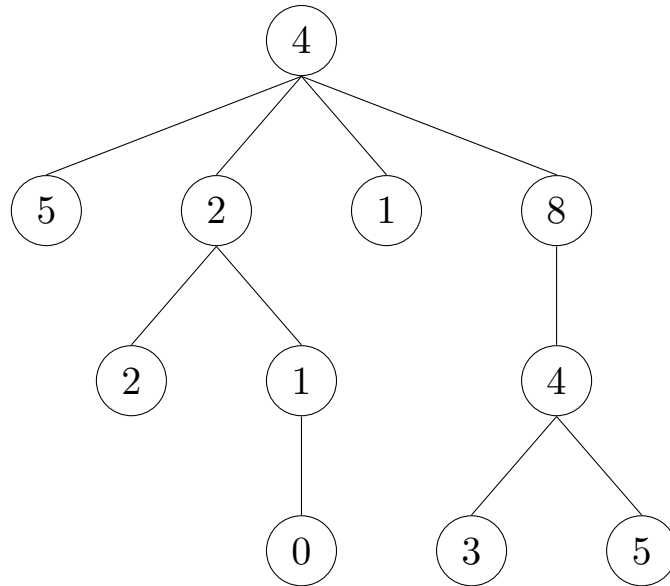
If you want, start off with an icebreaker of your choice!

Also, there is a link to a feedback form at the bottom of this worksheet! It would be amazing if you could highlight this/fill it out yourself so we can improve the worksheets in the future!

- Mini-lecture: Trees [3 min]
- Q1: Factor Tree [6 min]
- Q2: Even Square Tree [11 min]
- Q3: Pompom [8 min]
- Q4: Accumulate [12 min]
- Mini-lecture: iterators [5 min]
- Q4: WWPD Iterators [8 min]

1 Trees

Trees are a kind of recursive data structure. Each tree has a **root label** (which is some value) and a sequence of **branches**. Trees are “recursive” because the branches of a tree are trees themselves! A typical tree might look something like this:



This tree's root label is 4, and it has 4 branches, each of which is a smaller tree. The 6 of the tree's **subtrees** are also **leaves**, which are trees that have no branches.

Trees may also be viewed **relationally**, as a network of nodes with parent-child relationships. Under this scheme, each circle in the tree diagram above is a node. Every non-root node has one parent above it and every non-leaf node has at least one child below it.

Trees are represented by an abstract data type with a `tree` constructor and `label` and `branches` selectors. The `tree` constructor takes in a label and a list of branches and returns a tree. Here's how one would construct the tree shown above with `tree`:

```

tree(4,
    [tree(5),
     tree(2,
         [tree(2),
          tree(1,
              [tree(0)])]),
     tree(1),
     tree(8,
         [tree(4,
             [tree(3), tree(5)])])])])

```

The implementation of the ADT is provided here, but you shouldn't have to worry about this too much. (Remember the abstraction barrier!)

```

def tree(label, branches=[]):
    return [label] + list(branches)

def label(tree):
    return tree[0]

def branches(tree):
    return tree[1:] # returns a list of branches

```

Because trees are recursive data structures, recursion tends to be a very natural way of solving problems that involve trees.

- The **recursive case** for tree problems often involves recursive calls on the branches of a tree.
- The **base case** is often reached when we hit a leaf because there are no more branches to recurse on.

Teaching Tips

- Please make sure to check in with your students before mini-lecture so that you don't go over too much content that they already feel comfortable with.
- While it is typically true that you make the recursive calls on the branches of a tree and stop recursing when you reach a leaf, this is by no means always true, and you should make it clear that there will be exceptions to this rule of thumb.
- Common Misconceptions:
 - Students often have trouble with the idea that branches is a list of trees. Try to be specific when explaining, focusing on types. (Branches are lists, saving trees in them.)
 - * Try using the tree functions to build up different trees.
 - * Write out all the functions on the board and clearly define the types of the output and input.
 - Data Abstraction and Trees
 - * Although `t[0]` returns the label from the tree, students should be using `label(t)`. This is because `t` is not a list, it is a tree which is a data abstraction!
 - * It's important to explain why indexing branches (e.g. `branches(t)[0]`) doesn't violate an abstraction barrier (since branches returns a list of trees).
- The objectives for students are to:
 - Draw trees as graphical representations given Python code

- * Mention to students that empty branches `[]` is the default argument, so `tree(5)` is the same as `tree(5, [])`.
- * Emphasize variable types.
- * It may be helpful to mark pairs of parentheses to help in understand the nesting relationships.
 - Branches is a function that returns a list of trees.
 - Label values are numbers.

– Construct Python code given a graphical representation of a tree

1. Define the function `factor_tree` which returns a *factor tree*. Recall that in a factor tree, multiplying the leaves together is the prime factorization of the root, n .

```
def factor_tree(n):
    for i in _____:
        if _____:
            return tree(_____, _____)
    _____

    for i in range(2, n):
        if n % i == 0:
            return tree(n, [factor_tree(i), factor_tree(n // i)])
    return tree(n)
```

Suggested Time: 6 min

2. Write the function `even_square_tree`, which takes in a tree `t` and returns a new tree with only the even labels squared.

```
def even_square_tree(t):
    '''
    >>> t = tree(2, [tree(1), tree(4)])
    >>> even_square_tree(t)
    tree(4, [tree(1), tree(16)])
    '''
    _____

    if _____:
        return _____

    else:
        return _____

def even_square_tree(t):
    branches_s = [even_square_tree(b) for b in branches(t)]
    if label(t) % 2 == 0:
```

```
    return tree(label(t) * label(t), branches_s)
else:
    return tree(label(t), branches_s)
```

Students may get confused on this problem since there is no explicit base case defined, so please sure to emphasize that the `branches_s` operates as an implicit base case since if `t` is a leaf, there are no branches to iterate over and therefore `branches_s` is assigned to an empty list

Suggested Time: 11 min

2 Mutability

1. Draw the environment diagram that results from running the following code.

```
pom = [16, 15, 13]
pompom = pom * 2
pompom.append(pom[:])
pom.extend(pompom)
```

<https://goo.gl/ZU1V7h>

Suggested Time: 8 min

2. Given some list `lst` of numbers, mutate `lst` to have the accumulated sum of all elements so far in the list. If `lst` is a deep list, mutate it to similarly reflect the accumulated sum of all elements so far in the nested list. Your function should return an integer representing your “accumulated” sum (sum of all numbers in your list). You may not need all lines provided.

Hint: The **`isinstance`** function returns True for **`isinstance(l, list)`** if `l` is a list and False otherwise.

```
def accumulate(lst):
    '''
    >>> l = [1, 5, 13, 4]
    >>> accumulate(l)
    23
    >>> l
    [1, 6, 19, 23]
    >>> deep_l = [3, 7, [2, 5, 6], 9]
    >>> accumulate(deep_l)
    32
    >>> deep_l
    [3, 10, [2, 7, 13], 32]
    '''
    sum_so_far = 0
    for _____:

        _____

        if isinstance(_____, list):

            inside = _____

            _____

        else:

            _____

            _____

    _____
```

```
def accumulate(lst):
    sum_so_far = 0
    for i in range(len(lst)):
        item = lst[i]
        if isinstance(item, list):
            inside = accumulate(item)
            sum_so_far += inside
        else:
            sum_so_far += item
            lst[i] = sum_so_far
    return sum_so_far
```

Teaching Tips

- To keep track of the accumulated sum we need to create a variable that we update every time we see a new element.
- Make sure to emphasize the distinction between `for item in lst` and `for i in range(len(lst))`.
 - We need `i` in order to mutate the list. Why does using `for item in lst` not work when mutating? (Answer: because we're using a copy of the element, not modifying the original list).
 - Perhaps allow your students to first make the mistake of using the former, so that they may realize this difference on their own. Granted, if they aren't able to catch this on their own, do nudge them in the right direction.
- Why do we need a conditional in the for loop? What do we do when we have a nested list?
 1. Integers: For integers we just add the value to the ongoing sum and then mutate the current index of the list to be the cumulative sum
 2. Lists: We need to break down the list and get the values, both so that we can update them and so that we can add it into our sum. However, we don't know how many levels of nesting we have in our list
 - We could have something like `[1, [2, [3]]]`, so we need a function that will sum up the values from a potentially nested list. Do we have a function that does this?
 - **Emphasize the recursive leap of faith when calling accumulate on the inner list**
 - Remind students that they can use `isinstance` to check if an element is a list.
- We return the accumulated sum of the list which includes all values, even the nested ones because of the recursive call.

Suggested Time: 10 min

1. What Would Python Display? If you think nothing will be displayed, write N/A.

```
>>> a = [1, 2, 3]
>>> x = iter(a)
```

N/A

```
>>> next(x)
```

1

```
>>> next(x)
```

2

```
>>> y = iter(a)
>>> next(y)
```

1

```
>>> next(x)
```

3

```
>>> next(x)
```

StopIteration Error

```
>>> z = iter(y)
>>> next(z)
```

2

```
>>> [next(y), next(y), next(z)]
```

[2, 3, 3]

```
>>> a = iter(filter(lambda x: x % 2, map(lambda x: x - 1, range(10))))
>>> next(a)
```

-1

```
>>> reduce(lambda x, y: x + y, a)
```

Suggested Time: 8 min